Identificação do Comportamento Térmico de uma CPU: Ajuste de Curvas via RBF e Mínimos Quadrados

Oct 31, 2026 · @João Víctor Medeiros

Campo	Informação
Aluno	João Víctor Taveira Medeiros
Matrícula	20230071762
Professor	Joilson B. A. Rego
Disciplina	ECT3401 Computação Numérica, Unidade II
Palavras-chave	Ajuste de curvas · Funções de Base Radial · Mínimos Quadrados · Temperatura da CPU
Repositório	github.com/jvmedeiros0901/ajuste_de_curvas

1. Contexto e Motivação

Este projeto obtém modelos matemáticos para a temperatura do processador de um notebook a partir de medições feitas nele mesmo, e compara qual modelo representa melhor o sistema.

Todo processador esquenta quando trabalha. Controlar essa temperatura é um problema central da Engenharia de Computação: o firmware decide a velocidade da ventoinha, e o processador reduz a própria potência quando esquenta demais. Para projetar esse controle, ou prever quando o desempenho vai cair, é preciso um modelo de como a temperatura responde à carga de trabalho.

Aqui o próprio notebook foi tratado como o sistema a ser identificado:

- **Entrada u:** carga na CPU. Um script liga um cálculo pesado em todos os núcleos por 30 s (u = 1) e desliga por 30 s (u = 0), formando uma onda quadrada.
- **Saída 1:** temperatura do processador, em °C.
- **Saída 2:** velocidade da ventoinha, em rpm.

O sensor de temperatura mede em graus inteiros, então os dados vêm com ruído de arredondamento, além das variações naturais. Obter um modelo a partir dos dados, sem conhecer as equações internas do sistema, se chama **identificação de sistemas**. A forma

do modelo segue o enunciado:

y[n] = f(y[n−2], y[n−1], u[n], u[n−1])

Em palavras: a temperatura agora depende das duas temperaturas anteriores, que guardam a “memória” do aquecimento, e da carga de agora e da anterior. O ajuste foi feito com funções de base radial (RBF) e com o método dos mínimos quadrados (MMQ), e os dois foram avaliados com as mesmas métricas.

2. Objetivos

Todos os itens do enunciado foram atendidos, com dados reais coletados no próprio notebook. Além deles, o modelo do MMQ também foi resolvido por gradiente descendente, para conferir os pesos, e testado em simulação livre.

Ref.	Objetivo	Situação	Onde está no código
O0	Coletar dados reais de entrada e saída da CPU	Atendido	`coletar_cpu.py`
O1	Ajuste via Funções de Base Radial (2,0 pts)	Atendido	`matriz_rbf`, `centros_uniformes`
O2	Erro Médio Quadrático, EMQ (1,0 pt)	Atendido	`emq` (e também `rmse`, `mae`)
O3	Regressão linear entre dados reais e ajustados (1,0 pt)	Atendido	`regressao_linear`
O4	Coeficiente de correlação de Pearson (2,0 pts)	Atendido	`pearson`
O5	Repetir o processo com o MMQ (2,0 pts)	Atendido	`matriz_regressores`, `resolver_equacao_normal`
O6	Identificar os parâmetros livres do modelo	Atendido	pesos w0 a w4 impressos pelo `main.py`
Extra	Conferir os pesos por gradiente descendente e simular o modelo sozinho	Feito	`gradiente_descendente`, `simular`

3. Metodologia

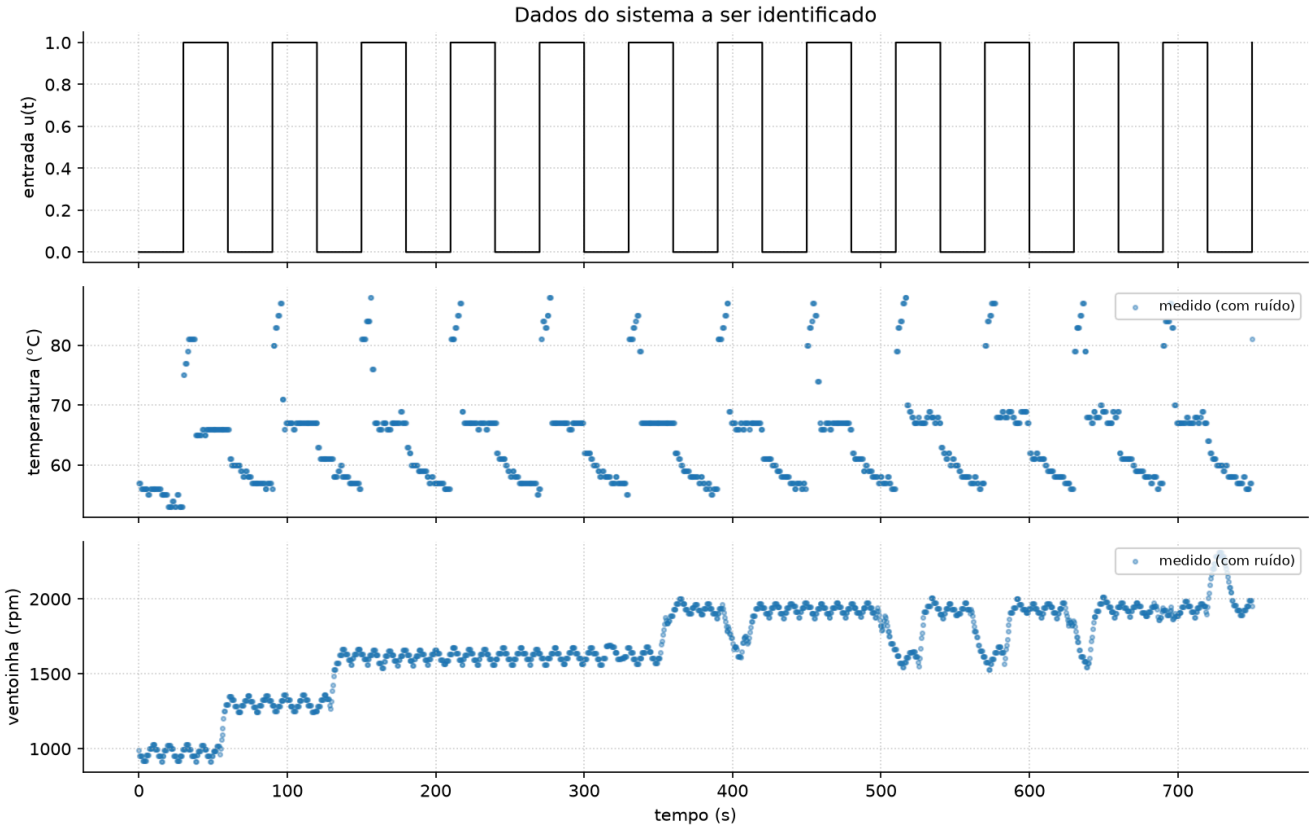
O programa foi feito em Python, com NumPy para as contas com matrizes e Matplotlib para os gráficos. As métricas, a regressão linear e o gradiente descendente foram escritos com as fórmulas vistas em aula, sem funções prontas de estatística.

3.1 Coleta dos dados

Os dados foram coletados por `coletar_cpu.py` num notebook Dell com processador Intel Core i7-1255U (12 threads), Ubuntu, ligado na tomada.

Item	Valor
Entrada u	12 processos fazendo contas sem parar (u = 1) ou parados (u = 0)
Alternância	30 s com carga, 30 s sem carga, começando sem carga
Temperatura	sensor `coretemp`, “Package id 0”, resolução de 1 °C
Ventoinha	sensor `dell_smm`, `fan1_input`, em rpm
Amostragem	uma leitura a cada 0,5 s
Duração	750 s (12,5 min), 1501 amostras

O Linux expõe os sensores como arquivos de texto em `/sys/class/hwmon`. O script procura cada sensor pelo nome do chip, porque a numeração das pastas pode mudar a cada inicialização.



A figura 1 já mostra o comportamento que os modelos precisam reproduzir:

- **Temperatura:** ao ligar a carga, ela dispara para cerca de 85 °C (média de 84,8 °C nos primeiros 5 s) e, poucos segundos depois, cai para um patamar de cerca de 67 °C. Ao desligar, desce em degraus de 1 °C até cerca de 57 °C.
- **Ventoinha:** muda devagar e em degraus (cerca de 1000, 1300, 1600 e 1950 rpm), subindo ao longo dos minutos e caindo um pouco em alguns períodos sem carga.

3.2 A ideia comum aos dois modelos

Os dois modelos são “lineares nos pesos”: a saída prevista é uma soma de funções, cada uma multiplicada por um peso. Empilhando todos os instantes, isso vira uma multiplicação de matriz por vetor:

$$\hat{y} = \Phi \cdot w$$

Cada linha de Φ (a matriz de projeto) é um instante, e cada coluna é uma função de base calculada naquele instante. Os pesos que deixam o erro quadrático mínimo saem da **equação normal** (equação 6.4 da apostila):

$$\Phi^T\Phi \cdot w = \Phi^T \cdot y \quad \Rightarrow \quad w = \Phi^+ \cdot y$$

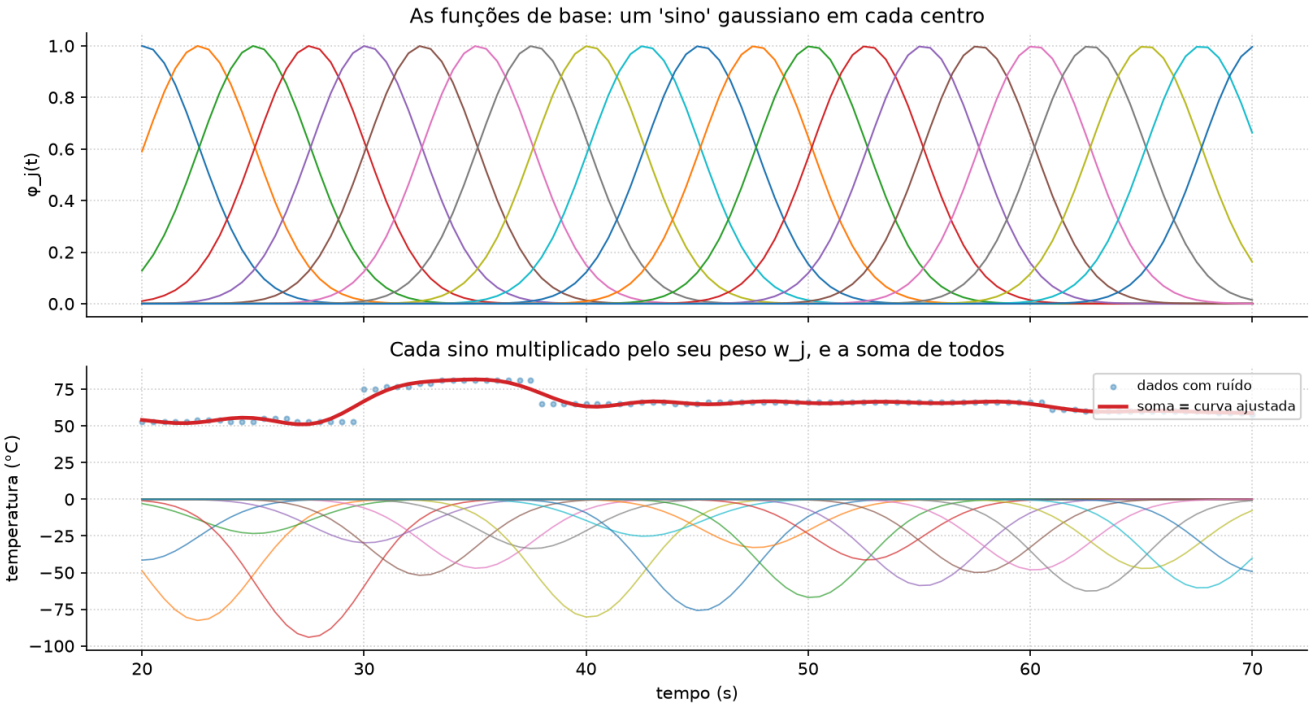
No código, `resolver_equacao_normal` usa a pseudo-inversa Φ^+ (`np.linalg.pinv`), que dá o mesmo resultado sem inverter $\Phi^T\Phi$ diretamente e é mais estável quando essa matriz é quase singular, como na RBF. **O que muda de um modelo para o outro são só as colunas de Φ.**

3.3 Modelo 1: Funções de Base Radial (RBF)

A RBF desenha a curva ao longo do tempo somando “sinos” gaussianos:

$$\hat{y}(t) = w_0 + w_1 \cdot \varphi_1(t) + \ldots + w_m \cdot \varphi_m(t), \quad \text{com} \quad \varphi_j(t) = \exp\left(-(t - c_j)^2 / 2\sigma^2\right)$$

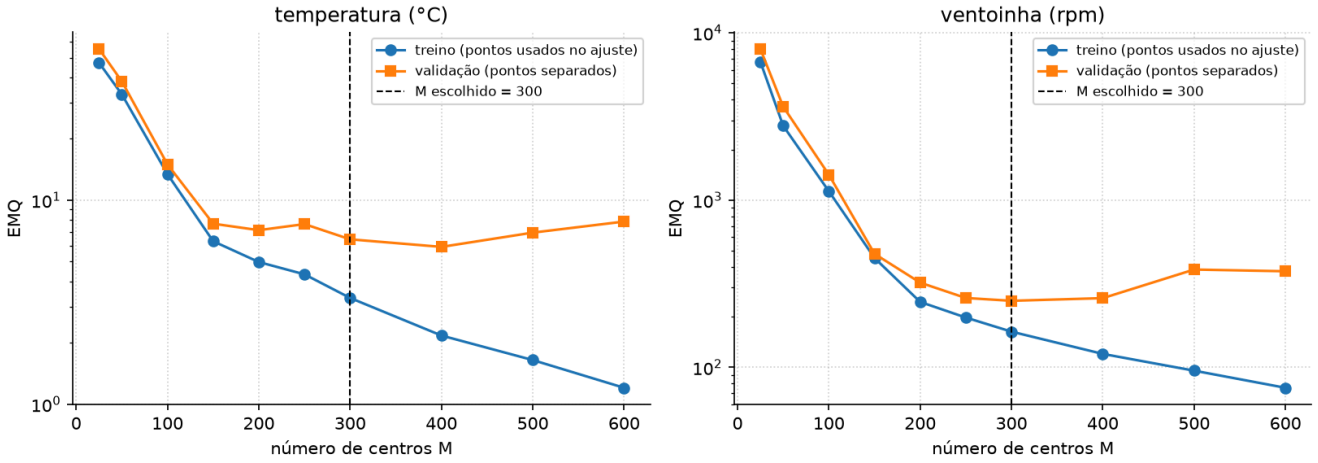
Os centros c_j foram espalhados igualmente de 0 a 750 s, e a largura σ é o espaçamento entre centros vizinhos. A figura 2 mostra um trecho de 50 s: em cima, os sinos; embaixo, cada sino multiplicado pelo seu peso e a soma, que é a curva ajustada.



Como a curva passa pelo meio dos pontos, ela funciona como um **filtro**: suaviza os degraus de 1 °C e as oscilações da leitura.

3.4 Escolha do número de gaussianas M

Para escolher M, o programa sorteia 20% dos pontos e os esconde do ajuste (validação). O modelo é ajustado com os outros 80% e testado nos pontos escondidos, para M de 25 a 600.



- **M até 100:** σ passa de 7 s. Os sinos são largos demais para acompanhar o salto de temperatura, que acontece em menos de 2 s, e o erro é alto nos dois conjuntos (subajuste).
- **M acima de 400:** o erro de treino continua caindo, mas o de validação sobe. A curva começa a seguir o ruído (sobreajuste).
- **M = 300 (σ = 2,51 s):** tem o menor erro de validação da ventoinha (249,9 rpm²) e fica perto do mínimo da temperatura (6,43 °C², contra 5,91 °C² com M = 400). É o modelo mais simples perto do mínimo nos dois sinais.

Com M escolhido, o ajuste final usa todos os pontos e tem 301 pesos por sinal.

3.5 Modelo 2: MMQ com o modelo dinâmico do enunciado

O segundo modelo é a forma linear do modelo pedido no enunciado:

$$\hat{y}[n] = w_0 + w_1 \cdot y[n-1] + w_2 \cdot y[n-2] + w_3 \cdot u[n] + w_4 \cdot u[n-1]$$

As colunas de Φ são uma coluna de 1s, a saída atrasada de um e de dois passos e a carga atual e a anterior (`matriz_regressores`). Os pesos w0 a w4 são os **parâmetros livres** do modelo. Eles foram calculados só com os primeiros 70% do tempo (t < 525 s, treino), e os últimos 225 s ficaram para teste. O mesmo modelo foi aplicado à ventoinha.

3.6 Previsão de um passo e simulação livre

- **Previsão de um passo** (`prever_um_passo`): para prever a temperatura no próximo meio segundo, usa os valores **medidos** anteriores. É como fazer uma prova com cola.
- **Simulação livre** (`simular`): só os dois primeiros valores vêm dos dados. Depois, o modelo usa as suas **próprias** previsões e recebe apenas a carga u. É a prova sem cola, e mostra se o modelo aprendeu a dinâmica térmica.

3.7 Conferência por gradiente descendente

Os pesos do MMQ também foram calculados de forma iterativa, partindo de w = 0 e dando passos no sentido em que o erro diminui:

$$\mathbf{w \leftarrow w - \eta \cdot \nabla J, \quad com \quad \nabla J = (2/N) \cdot \Phi^T(\Phi w - y)}$$

Antes, cada coluna foi padronizada por Z-score, z = (valor − média) / desvio-padrão. A taxa de aprendizado foi η = 1/λmax, em que λmax é o maior autovalor de $(2/N)\Phi^T\Phi$. O método só converge se η < 2/λmax, então essa escolha fica na metade do limite. O laço para quando o tamanho do gradiente fica abaixo de 10^{-6}.

3.8 Métricas

Para cada modelo, o erro de cada ponto é e = medido − ajustado.

Métrica	Fórmula	O que diz
EMQ (erro médio quadrático)	média de e^2	Erro típico ao quadrado (°C² ou rpm²); pesa mais os erros grandes
RMSE	raiz do EMQ	O mesmo, na unidade do sinal (°C ou rpm)
MAE (erro médio absoluto)	média de |e|	Erro típico, sem exagerar os erros grandes
r de Pearson	$\Sigma(a-\bar{a})(b-\bar{b}) / \sqrt{(\Sigma(a-\bar{a})^2 \cdot \Sigma(b-\bar{b})^2)}$	De −1 a 1: quanto os pontos medido × ajustado formam uma reta
Regressão linear	medido ≈ inclinação · ajustado + intercepto	Inclinação 1 e intercepto 0 indicam um ajuste sem tendência

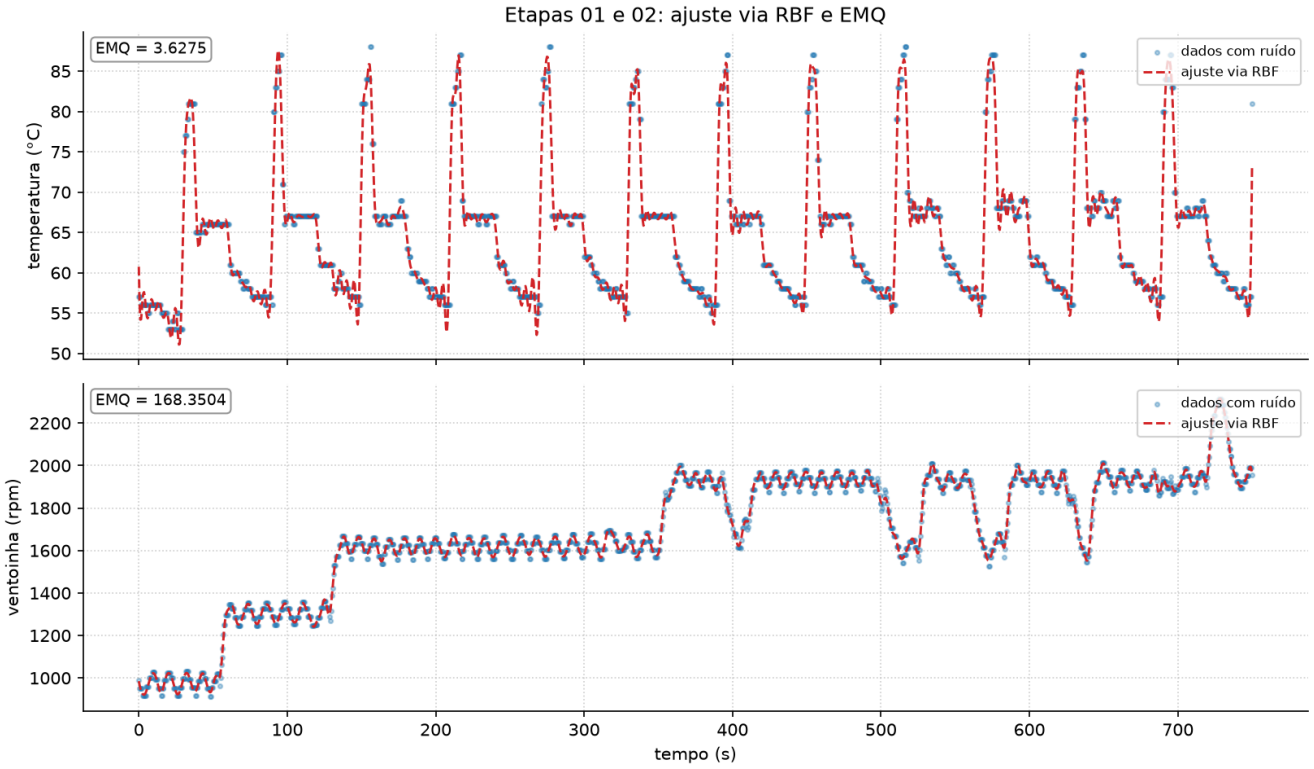
3.9 Caminho do programa

1. `coletar_cpu.py` alterna a carga, lê os sensores e grava `dados/dados_cpu.csv`.
2. `python main.py cpu` chama `carregar_dados` e desenha a figura 1.
3. Testa M de 25 a 600 com os pontos de validação e desenha a figura 3.
4. Ajusta a RBF com M = 300, calcula as métricas e desenha as figuras 2, 4 e 5.
5. Monta Φ do modelo dinâmico, resolve a equação normal no treino e imprime os pesos.
6. Faz a previsão de um passo e a simulação livre e desenha as figuras 6, 7 e 8.
7. Refaz os pesos por gradiente descendente, compara e desenha a figura 9. Tudo fica salvo em `resultados_cpu/`.

4. Resultados

A RBF teve o menor erro nos dois sinais (r = 0,973 na temperatura). O MMQ chegou perto na previsão de um passo, mas, andando sozinho, não reproduziu o pico do turbo nem os degraus da ventoinha. Todos os números abaixo são impressos por `python main.py cpu` e ficam salvos em `resultados_cpu/resultados.txt`.

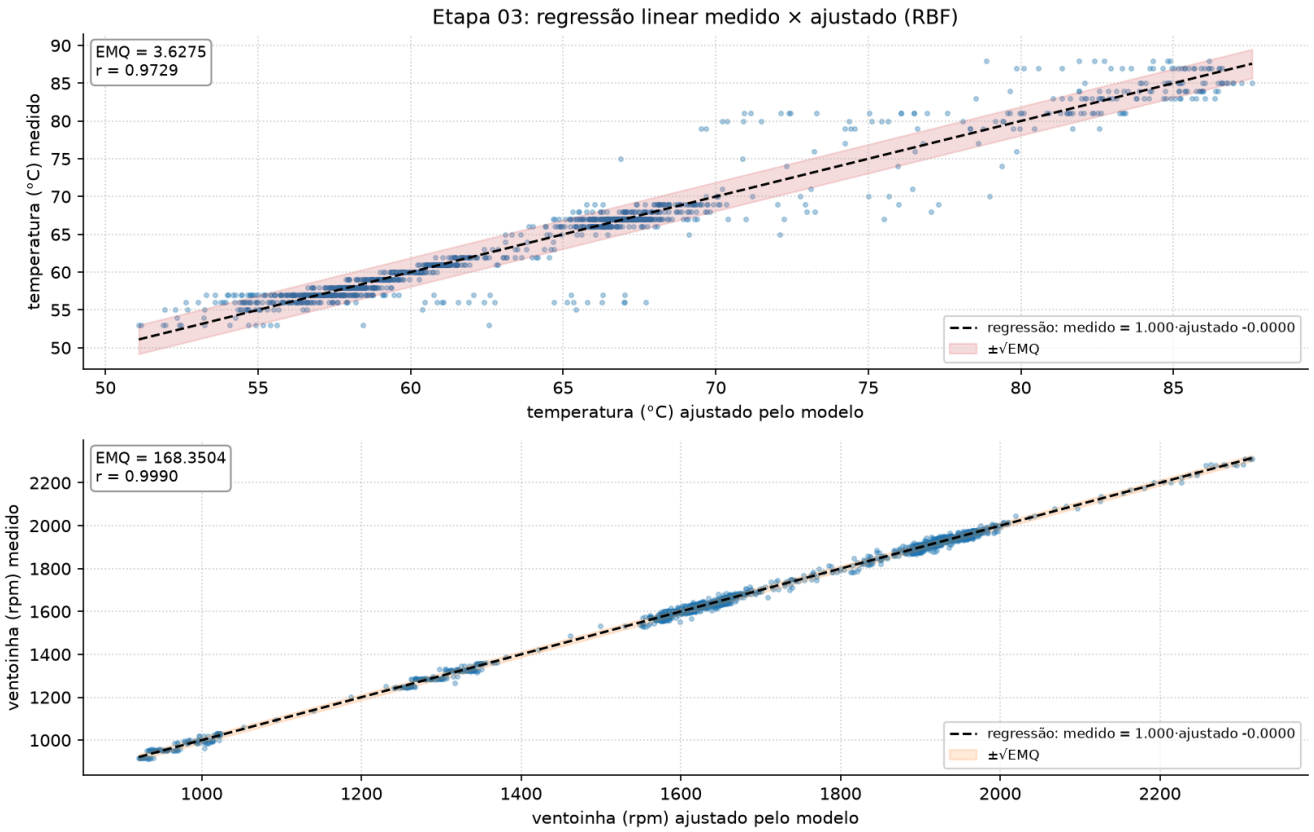
4.1 Etapas 01 e 02: ajuste via RBF e EMQ



Sinal	EMQ	RMSE	MAE	r
Temperatura	3,63 °C²	1,90 °C	1,01 °C	0,9729
Ventoinha	168,4 rpm²	13,0 rpm	10,7 rpm	0,9990

A curva segue os picos, os patamares e as descidas da temperatura com erro típico de 1 °C, o tamanho de um degrau do sensor. O erro maior aparece logo antes de cada subida: a curva afunda alguns graus e depois sobe. Isso acontece porque uma soma de sinos suaves não consegue dar um salto de 25 °C em 1 s sem “balançar” em volta dele. Na ventoinha, que muda devagar, o ajuste é quase perfeito.

4.2 Etapa 03: regressão linear e correlação de Pearson



A reta de regressão deu **medido = 1,000 · ajustado + 0,0000** nos dois sinais, ou seja, o ajuste não fica sistematicamente acima nem abaixo dos dados. Isso sempre acontece num ajuste por mínimos quadrados com coluna de 1s, então quem mede a qualidade é o r. Elevando ao quadrado, r^2 diz quanto da variação dos dados o modelo explica: 94,7% na temperatura e 99,8% na ventoinha.

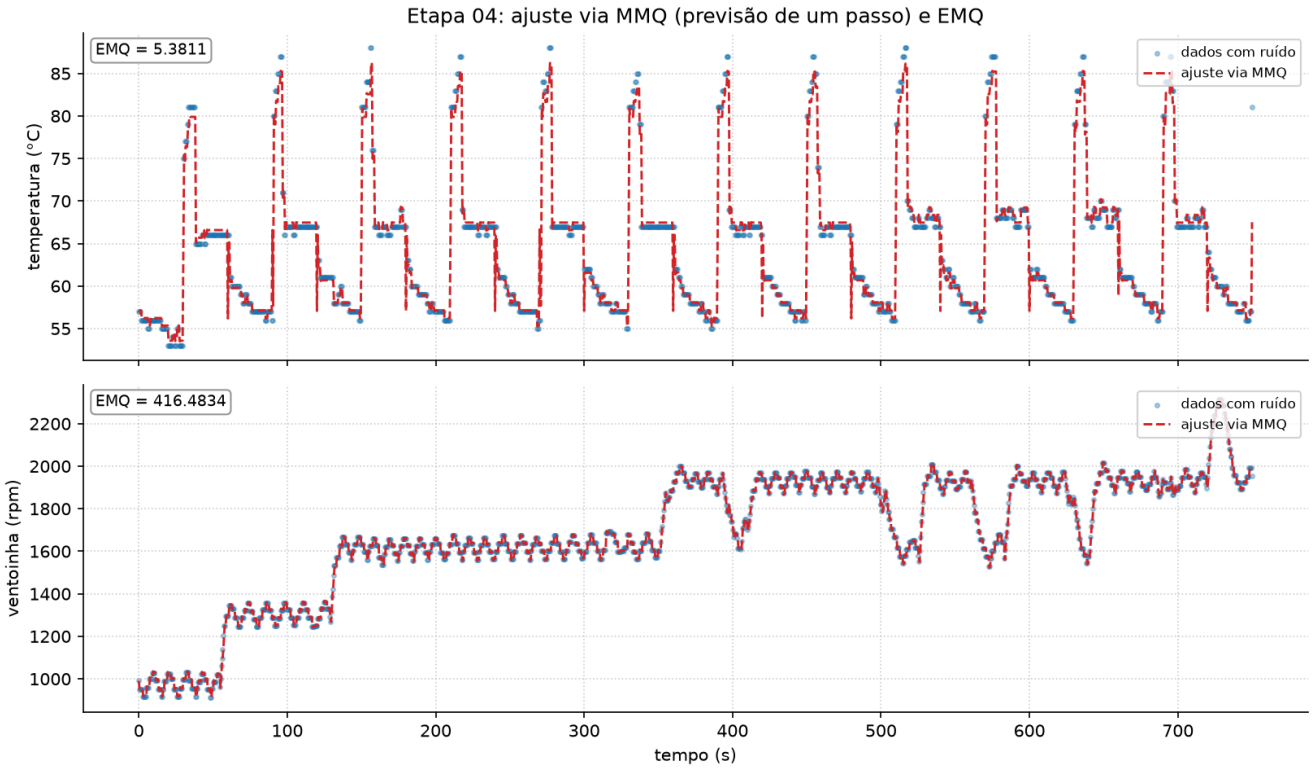
4.3 Etapa 04: ajuste via MMQ

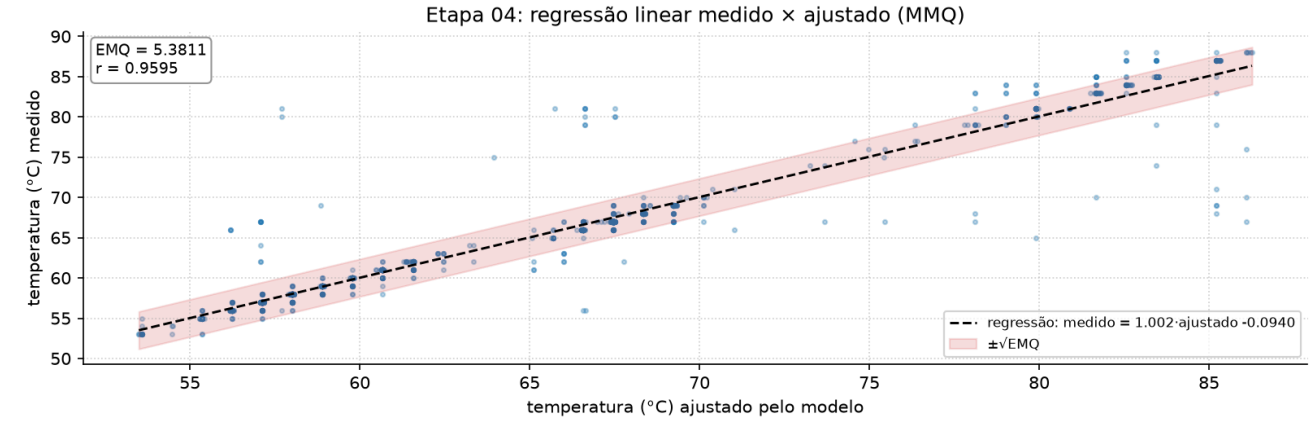
Parâmetros livres identificados pela equação normal, com os dados de treino:

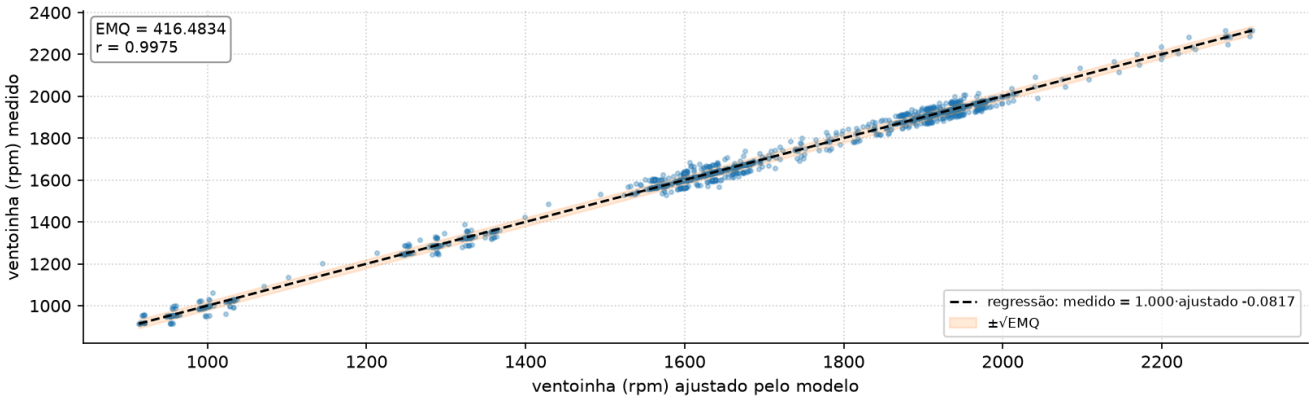
Peso	Multiplica	Temperatura	Ventoinha
w0	1 (constante)	6,50831	8,06627
w1	saída em n−1	0,92711	1,10803
w2	saída em n−2	−0,03909	−0,11280
w3	u[n]	10,37949	−4,30937
w4	u[n−1]	−8,92262	4,46355

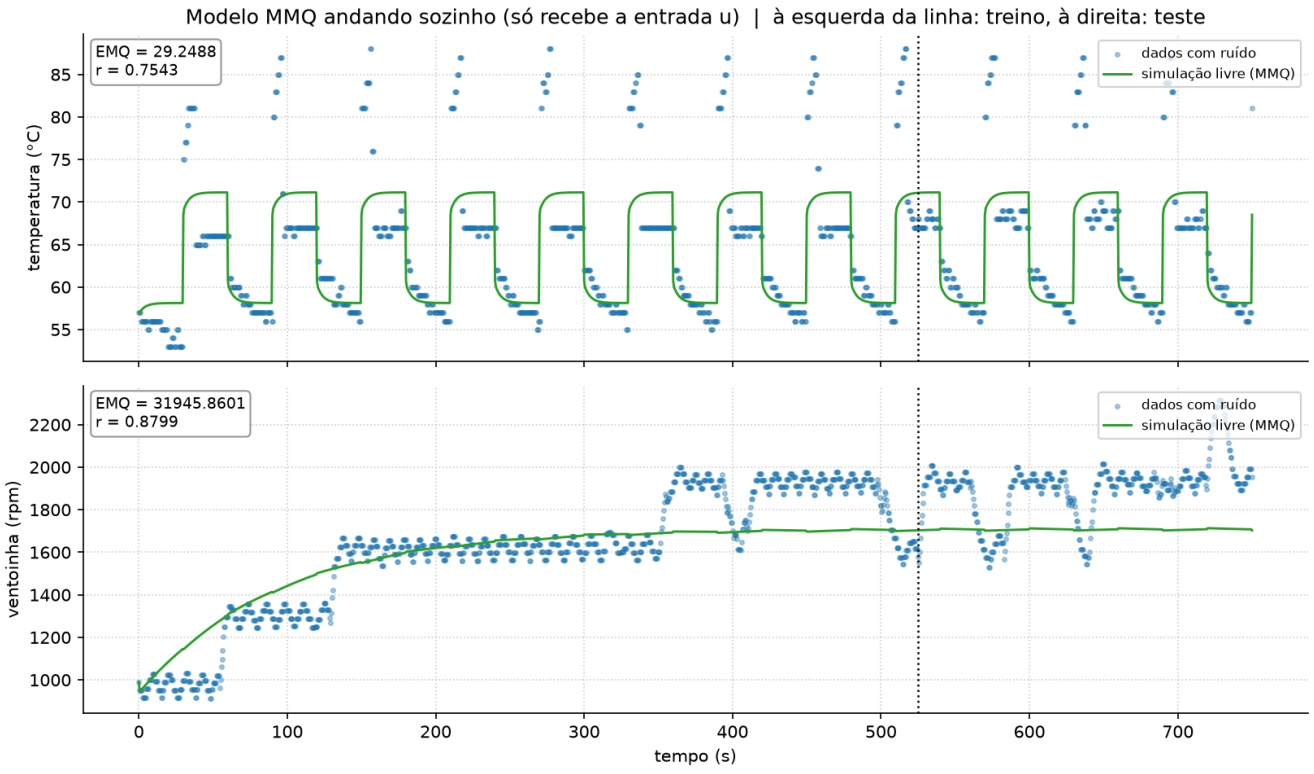
Os pesos têm significado físico. Fazendo y[n] = y[n−1] = y[n−2] na fórmula, dá para calcular onde o modelo se estabiliza:

- **Temperatura:** 58,1 °C sem carga e 71,1 °C com carga, com constante de tempo de 4,0 s. Os valores medidos foram cerca de 57 °C em repouso, pico de 84,8 °C e patamar de 67,1 °C. O modelo ficou com um valor no meio do pico e do patamar.
- **Ventoinha:** w1 + w2 = 0,995, muito perto de 1. Isso significa que o modelo quase só repete o valor anterior, com constante de tempo de 93 s, e se estabiliza perto de 1700 rpm com ou sem carga. O modelo concluiu que a ventoinha não responde diretamente à carga do momento.









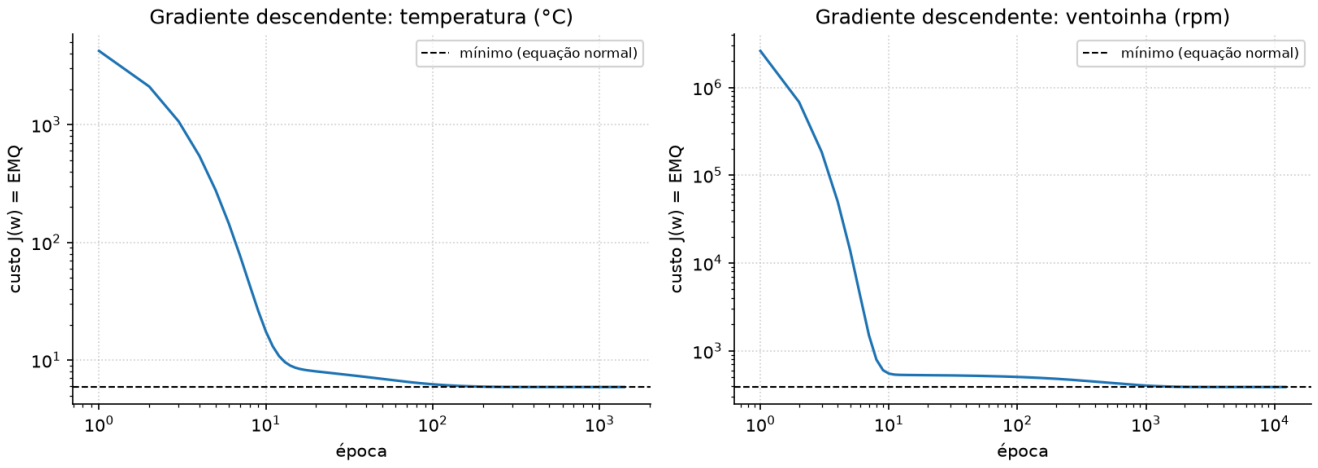
Uso do modelo	Trecho	EMQ temperatura	r temperatura	EMQ ventoinha	r ventoinha
Previsão de um passo	todos os pontos	5,38 °C²	0,9595	416 rpm²	0,9975

Uso do modelo	Trecho	EMQ temperatura	r temperatura	EMQ ventoinha	r ventoinha
Previsão de um passo	só teste (t ⩾ 525 s)	4,16 °C²	0,9658	483 rpm²	0,9882
Simulação livre	todos os pontos	29,25 °C²	0,7543	31 946 rpm²	0,8799
Simulação livre	só teste (t ⩾ 525 s)	25,51 °C²	0,7700	57 382 rpm²	0,6487

Na previsão de um passo, os erros no teste são parecidos com os de todos os pontos, então o modelo não decorou o treino. Na simulação livre (figura 8), a temperatura vira uma onda quadrada arredondada entre 58 e 71 °C: acerta o ritmo, mas não tem o pico de 85 °C. A ventoinha simulada sobe devagar até 1700 rpm e fica parada ali, sem os degraus.

4.4 Conferência por gradiente descendente

Sinal	Taxa η	Condicionamento	Épocas	Maior diferença nos pesos
Temperatura	0,145	108	1404	$2{,}2 \times 10^{-5}$
Ventoinha	0,240	871	12 183	$1{,}9 \times 10^{-6}$



O gradiente descendente chegou nos mesmos pesos da equação normal. A ventoinha precisou de quase 9 vezes mais épocas porque o condicionamento foi 8 vezes maior. Seus valores em n−1 e n−2 são quase iguais, o que deixa a “tigela” do erro achatada numa direção, e a descida fica lenta. É exatamente o efeito previsto no mini-projeto da disciplina.

4.5 Comparação final

Modelo	EMQ temperatura	r temperatura	EMQ ventoinha	r ventoinha
RBF, 300 gaussianas	3,63 °C²	0,9729	168 rpm²	0,9990
MMQ, previsão de um passo	5,38 °C²	0,9595	416 rpm²	0,9975
MMQ, simulação livre	29,25 °C²	0,7543	31 946 rpm²	0,8799

4.6 Interpretação

A RBF é o melhor filtro. Com 301 pesos, cada sino cuida de um trecho de cerca de 2,5 s, e a curva segue cada pico e cada degrau. Mas ela só descreve o que já foi medido: não usa a carga u e não prevê a temperatura de amanhã.

O MMQ dá um modelo de verdade, com significado físico, mas linear demais para este processador. Com só 5 pesos, ele descobriu que a temperatura de repouso é cerca de 58 °C, que a carga a leva para perto de 71 °C e que isso leva poucos segundos. O que ele não representa é o pico: o processador começa na potência máxima do turbo e, poucos segundos depois, o controle de energia da plataforma reduz a potência, por limite de potência ou de temperatura. Essa é uma regra do firmware, não uma resposta proporcional à carga. Os pesos w3 = 10,4 e w4 = −8,9, de sinais opostos, até dão um pequeno empurrão extra no instante em que a carga liga, mas o ajuste de um passo não chega nem perto do pico de 85 °C.

A ventoinha mostra o limite do modelo com mais clareza. Ela é controlada pelo firmware por faixas de temperatura, em degraus, e responde ao calor acumulado ao longo de minutos, não à carga do momento. O MMQ percebeu isso (w1 + w2 ≈ 1 e pesos de u quase nulos) e, por isso, a simulação livre virou uma curva lenta e plana.

A previsão de um passo engana. Ela tem r = 0,96 e 0,998 porque copia o valor medido anterior. A simulação livre, que testa o que o modelo aprendeu, cai para r = 0,75 e 0,88.

4.7 Avaliação crítica

O ponto mais forte do projeto é usar dados reais, coletados e reproduzíveis com um comando. Os resultados são explicados pelo funcionamento do hardware, e não apenas pelas métricas. A simulação livre e a separação treino/teste evitam conclusões tiradas só de números que parecem bons.

As limitações que encontrei:

- **Uma única coleta de 12,5 min.** A temperatura ambiente e o que mais rodava no notebook não foram controlados. O notebook começou já aquecido por um teste anterior, e a ventoinha ainda subia no fim da coleta.
- **Sensor de 1 °C.** O "ruído" é principalmente arredondamento, diferente de um ruído aleatório clássico.
- **Tempo das amostras.** O script agenda as leituras com `sleep` em Python e grava o instante planejado, não o instante real. Pequenos atrasos não foram medidos.
- **Última amostra.** Em t = 750 s a carga acabou de ligar, e há um único ponto do novo ciclo (81 °C), que puxa o final das curvas.
- **Modelo linear.** Para representar o turbo e os degraus da ventoinha seria preciso um modelo não linear, por exemplo uma RBF aplicada aos valores anteriores (y[n−1], y[n−2], u) em vez do tempo, ou usar como entrada a potência ou a frequência da CPU em vez de liga/desliga.

5. Cronograma do Projeto

Semana	Atividade	Entregável
1	Estudo da teoria (MMQ, equação normal, RBF, gradiente descendente) e criação do repositório	`dados.py`, README
2	Ajuste via RBF, escolha de M e métricas, validados com os dados do enunciado	`modelos.py`, `metricas.py`
3	Script de coleta, coleta dos dados da CPU, MMQ, simulação livre e gradiente descendente	`coletar_cpu.py`, `dados_cpu.csv`, figuras 1 a 9
4	Redigir relatório e slides da apresentação	Relatório final

6. Recursos Necessários

Um computador com Linux que exponha os sensores de temperatura da CPU e da ventoinha em `/sys/class/hwmon` (aqui, os drivers `coretemp` e `dell_smm`), ligado na tomada. O programa usa Python 3.11 ou mais novo, NumPy e Matplotlib, e o código está versionado no GitHub. Na pasta do projeto:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python coletar_cpu.py   # coleta nova (cerca de 12 min), opcional
python main.py cpu      # análise
```

A análise leva poucos segundos e salva as nove figuras e o arquivo `resultados.txt` em `resultados_cpu/`. Outros notebooks podem ter sensores com nomes diferentes; nesse caso, basta ajustar os nomes em `coletar_cpu.py`.

7. Referências Bibliográficas

1. REGO, Joilson B. A. *Computação Numérica: Projeto Unidade II*. Enunciado do projeto, ECT3401 Computação Numérica, UFRN.
2. REGO, Joilson B. A. *Capítulo 6: Ajuste de Curvas*. Notas de aula, ECT3401 Computação Numérica, UFRN. Seções 6.1 (regressão por mínimos quadrados) e 6.2 (funções de base radial).
3. REGO, Joilson B. A. *Mini projeto: Regressão via RBF (Gaussianas)* e *Regressão Linear Multivariada*. Notebooks da disciplina, ECT3401, UFRN.
4. LJUNG, Lennart. *System Identification: Theory for the User*. 2. ed. Prentice Hall, 1999.
5. BISHOP, Christopher M. *Pattern Recognition and Machine Learning*. Springer, 2006. Capítulo 3 (modelos lineares de regressão e funções de base).
6. THE LINUX KERNEL. *Hardware Monitoring Kernel Drivers*: coretemp e dell-smm-hwmon. Documentação do kernel Linux.
7. HARRIS, Charles R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357–362, 2020.
8. HUNTER, John D. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90–95, 2007.

Apresentado em cumprimento aos requisitos para obtenção da nota referente à Unidade II da disciplina ECT3401: Computação Numérica.